CampusLink Database Plan and Schema

Exported from the project Markdown document

CampusLink Database Plan and Schema

Overview

CampusLink should use a relational database, preferably PostgreSQL. The app already has clear structured domains: users, verification, marketplace listings, wallet, orders, billing, subscriptions, and messaging. A relational model fits the current frontend well and will make it easier to move from local storage and mock state into persistent backend data.

Recommended stack:

- Database: PostgreSQL
- Auth: Supabase Auth or equivalent
- File storage: object storage for KYC documents and listing media
- Payments: Paystack with backend verification and webhook handling

Core design rules:

- Store all money values in `kobo`, not float naira
- Use immutable wallet ledger records
- Snapshot listing data into order items
- Do not trust frontend payment success callbacks as source of truth
- Enforce verification and premium rules from the backend

Core domains

The system is made up of these main domains:

1. Identity and profiles
2. Student, expert, and institution verification
3. Marketplace categories and listings

4. Wallets, transactions, and payouts

5. Orders and order items

6. Premium subscription plans and subscriptions

7. Messaging and conversation history

8. Admin settings, moderation, and audit logging

Main tables

`profiles`

Stores application-level user data for students, experts, institutions, admins, and super admins.

Key fields:

- `id uuid primary key`
- `email citext unique not null`
- `full_name text not null`
- `user_type text not null`
- `role text default 'user'`
- `phone text`
- `location text`
- `bio text`
- `university text`
- `course text`
- `level text`
- `department text`
- `faculty text`
- `avatar_url text`
- `is_verified boolean default false`
- `verification_status text default 'not_started'`
- `is_authorized boolean default false`

- `is_premium boolean default false`
- `rating numeric(3,2) default 0`
- `reviews_count integer default 0`
- `completed_projects_count integer default 0`
- `earnings_total_kobo bigint default 0`
- `created_at timestamptz default now()`
- `updated_at timestamptz default now()`

Optional subtype fields:

- `expertise text[]`
- `hourly_rate_kobo bigint`
- `years_of_experience integer`
- `certifications text[]`
- `institution_name text`
- `position text`

`verification_requests`

Tracks student KYC and future expert or institution verification workflows.

Key fields:

- `id uuid primary key`
- `user_id uuid not null references profiles(id)`
- `verification_type text not null`
- `status text not null default 'pending'`
- `submitted_at timestamptz`
- `reviewed_at timestamptz`
- `reviewed_by uuid references profiles(id)`
- `rejection_reason text`
- `payload jsonb`
- `created_at timestamptz default now()`

`verification_documents`

Stores metadata for uploaded verification files.

Key fields:

- `id uuid primary key`
- `verification_request_id uuid not null references verification_requests(id)`
- `document_type text not null`
- `storage_path text not null`
- `file_name text not null`
- `mime_type text`
- `file_size_bytes bigint`
- `checksum text`
- `created_at timestamptz default now()`

`wallets`

One wallet per user.

Key fields:

- `id uuid primary key`
- `user_id uuid unique not null references profiles(id)`
- `currency_code text not null default 'NGN'`
- `available_balance_kobo bigint default 0`
- `ledger_balance_kobo bigint default 0`
- `status text default 'active'`
- `created_at timestamptz default now()`
- `updated_at timestamptz default now()`

`wallet_ledger_entries`

Immutable transaction log for wallet activity.

Key fields:

- `id uuid primary key`
- `wallet_id uuid not null references wallets(id)`
- `entry_type text not null`
- `direction text not null`
- `amount_kobo bigint not null`
- `currency_code text not null default 'NGN'`
- `status text not null`
- `reference text not null`
- `description text`
- `source_type text`
- `source_id uuid`
- `metadata jsonb`
- `created_at timestamptz default now()`

`payout_accounts`

Bank account details used for withdrawals.

Key fields:

- `id uuid primary key`
- `user_id uuid not null references profiles(id)`
- `bank_name text not null`
- `account_number text not null`
- `account_name text not null`
- `provider_recipient_code text`
- `is_default boolean default false`
- `is_verified boolean default false`
- `created_at timestamptz default now()`

`payment_attempts`

One table for all Paystack-related payment records.

Key fields:

- `id uuid primary key`
- `user_id uuid not null references profiles(id)`
- `purpose text not null`
- `provider text not null default 'paystack'`
- `provider_reference text unique not null`
- `provider_transaction_id text`
- `payment_method text`
- `amount_kobo bigint not null`
- `currency_code text default 'NGN'`
- `status text not null`
- `order_id uuid`
- `subscription_id uuid`
- `metadata jsonb`
- `initialized_at timestamptz`
- `confirmed_at timestamptz`
- `created_at timestamptz default now()`

`subscription_plans`

Admin-controlled premium plan catalog.

Key fields:

- `id text primary key`
- `name text not null`
- `price_kobo bigint not null`
- `currency_code text not null default 'NGN'`
- `interval text not null`
- `is_active boolean default true`
- `is_popular boolean default false`
- `features jsonb not null`

- `created_at timestamptz default now()`
- `updated_at timestamptz default now()`

Seed plans:

- `monthly`
- `yearly`

`subscriptions`

Stores user premium status and billing lifecycle.

Key fields:

- `id uuid primary key`
- `user_id uuid not null references profiles(id)`
- `plan_id text not null references subscription_plans(id)`
- `status text not null`
- `started_at timestamptz`
- `current_period_start timestamptz`
- `current_period_end timestamptz`
- `cancelled_at timestamptz`
- `latest_payment_attempt_id uuid`
- `created_at timestamptz default now()`

`marketplace_categories`

Normalized category catalog for all marketplace navigation and filtering.

Key fields:

- `id text primary key`
- `label text not null`
- `kind text not null`
- `sort_order integer default 0`
- `is_active boolean default true`

Suggested seed values:

- `all`
- `services`
- `goods`
- `assignment-help`
- `research`
- `design`
- `coding`
- `books`
- `gadgets`

`marketplace_listings`

Unified table for both service and goods listings.

Key fields:

- `id uuid primary key`
- `seller_id uuid not null references profiles(id)`
- `category_id text not null references marketplace_categories(id)`
- `seller_type text not null`
- `item_type text not null`
- `title text not null`
- `description text not null`
- `price_kobo bigint not null`
- `currency_code text not null default 'NGN'`
- `location text`
- `delivery_text text`
- `status text not null default 'draft'`
- `quantity_available integer`
- `fulfillment_mode text`
- `rating_avg numeric(3,2) default 0`

- `reviews_count integer default 0`
- `search_vector tsvector`
- `created_at timestamptz default now()`
- `updated_at timestamptz default now()`

`listing_media`

Media attached to listings.

Key fields:

- `id uuid primary key`
- `listing_id uuid not null references marketplace_listings(id)`
- `storage_path text not null`
- `alt_text text`
- `sort_order integer default 0`
- `created_at timestamptz default now()`

`orders`

High-level order record for checkout and payment.

Key fields:

- `id uuid primary key`
- `buyer_id uuid not null references profiles(id)`
- `status text not null`
- `payment_status text not null`
- `payment_method text not null`
- `payment_attempt_id uuid`
- `payment_reference text`
- `subtotal_kobo bigint not null`
- `service_fee_kobo bigint not null`
- `delivery_fee_kobo bigint not null`

- `total_kobo bigint not null`
- `currency_code text not null default 'NGN'`
- `delivery_name text not null`
- `delivery_email text not null`
- `delivery_phone text not null`
- `delivery_address text not null`
- `delivery_city text not null`
- `delivery_state text not null`
- `delivery_note text`
- `estimated_delivery_text text`
- `created_at timestamptz default now()`
- `updated_at timestamptz default now()`

`order_items`

Snapshot order lines copied from the listing at purchase time.

Key fields:

- `id uuid primary key`
- `order_id uuid not null references orders(id)`
- `listing_id uuid references marketplace_listings(id)`
- `seller_id uuid not null references profiles(id)`
- `item_type text not null`
- `title_snapshot text not null`
- `description_snapshot text`
- `seller_name_snapshot text`
- `category_snapshot text`
- `location_snapshot text`
- `delivery_snapshot text`
- `unit_price_kobo bigint not null`
- `quantity integer not null`

- `line_total_kobo bigint not null`
- `status text not null`
- `created_at timestamptz default now()`

`order_status_history`

Audit trail for order changes.

Key fields:

- `id uuid primary key`
- `order_id uuid not null references orders(id)`
- `status text not null`
- `changed_by uuid references profiles(id)`
- `note text`
- `created_at timestamptz default now()`

`conversations`

Messaging threads between users.

Key fields:

- `id uuid primary key`
- `conversation_type text not null`
- `created_by uuid not null references profiles(id)`
- `listing_id uuid references marketplace_listings(id)`
- `order_id uuid references orders(id)`
- `title text`
- `last_message_at timestamptz`
- `created_at timestamptz default now()`

`conversation_participants`

Tracks membership and read position in a conversation.

Key fields:

- `conversation_id uuid not null references conversations(id)`
- `user_id uuid not null references profiles(id)`
- `role text default 'member'`
- `last_read_message_id uuid`
- `joined_at timestamptz default now()`
- `muted_at timestamptz`

Primary key:

- `(conversation_id, user_id)`

`messages`

Message records for chat.

Key fields:

- `id uuid primary key`
- `conversation_id uuid not null references conversations(id)`
- `sender_id uuid not null references profiles(id)`
- `message_type text not null default 'text'`
- `body text`
- `reply_to_message_id uuid`
- `metadata jsonb`
- `created_at timestamptz default now()`
- `edited_at timestamptz`
- `deleted_at timestamptz`

`message_attachments`

Attachments linked to messages.

Key fields:

- `id uuid primary key`
- `message_id uuid not null references messages(id)`
- `storage_path text not null`
- `file_name text not null`
- `mime_type text`
- `file_size_bytes bigint`

`disputes`

Captures refunds, non-delivery, wrong item, damaged item, or service complaints.

Key fields:

- `id uuid primary key`
- `order_id uuid not null references orders(id)`
- `order_item_id uuid references order_items(id)`
- `opened_by uuid not null references profiles(id)`
- `reason text not null`
- `status text not null`
- `summary text`
- `resolution_note text`
- `resolved_by uuid references profiles(id)`
- `resolved_at timestamptz`
- `created_at timestamptz default now()`

`system_settings`

Global platform settings for admin-controlled configuration.

Key fields:

- `key text primary key`
- `value jsonb not null`
- `updated_by uuid references profiles(id)`

- `updated_at timestamptz default now()`

This table should store things like:

- maintenance mode
- registration enabled
- platform commission
- minimum withdrawal
- premium fee configuration if you want plans in settings format

`admin_audit_logs`

Tracks moderation, verification, and settings changes made by admins.

Key fields:

- `id uuid primary key`
- `actor_user_id uuid not null references profiles(id)`
- `action text not null`
- `target_type text not null`
- `target_id uuid`
- `payload jsonb`
- `created_at timestamptz default now()`

Relationships

One-to-one

- `profiles` -> `wallets`

One-to-many

- `profiles` -> `verification_requests`
- `verification_requests` -> `verification_documents`
- `profiles` -> `payout_accounts`

- `profiles` -> `payment_attempts`
- `profiles` -> `marketplace_listings`
- `profiles` -> `orders`
- `orders` -> `order_items`
- `orders` -> `order_status_history`
- `conversations` -> `messages`
- `messages` -> `message_attachments`
- `orders` -> `disputes`

Many-to-many

- `profiles` <-> `conversations` through `conversation_participants`

Recommended enums

Use PostgreSQL enums or strict text constraints for these values:

- `user_type`: `student`, `expert`, `institution`
- `role`: `user`, `admin`, `super_admin`
- `verification_status`: `not_started`, `pending`, `verified`, `rejected`
- `verification_type`: `student_kyc`, `expert_verification`, `institution_verification`
- `document_type`: `student_id`, `government_id`, `certificate`, `other`
- `wallet_status`: `active`, `restricted`, `suspended`
- `ledger_entry_type`: `deposit`, `withdrawal`, `payment`, `earning`, `refund`, `fee`, `hold`, `release`
- `ledger_direction`: `credit`, `debit`
- `payment_purpose`: `wallet_deposit`, `marketplace_checkout`, `subscription`, `withdrawal_payout`, `refund`
- `payment_method`: `wallet`, `card`, `transfer`, `ussd`, `bank`, `bank_transfer`
- `provider_status`: `initialized`, `pending`, `paid`, `failed`, `cancelled`, `refunded`
- `subscription_status`: `active`, `past_due`, `cancelled`, `expired`
- `listing_status`: `draft`, `active`, `paused`, `sold`, `archived`, `moderated`
- `item_type`: `service`, `goods`

- `order_status`: `pending_payment`, `paid`, `processing`, `fulfilled`, `completed`, `cancelled`, `disputed`, `refunded`
- `order_item_status`: `pending`, `processing`, `fulfilled`, `cancelled`, `refunded`, `disputed`
- `conversation_type`: `direct`, `group`, `order_support`
- `message_type`: `text`, `file`, `image`, `system`
- `dispute_reason`: `item_not_as_described`, `not_delivered`, `damaged`, `duplicate_charge`, `unauthorized_payment`, `service_abandoned`, `other`
- `dispute_status`: `open`, `under_review`, `resolved_refund`, `resolved_reject`, `resolved_credit`, `closed`

Important indexes

Profiles and verification

- `unique index on profiles(email)`
- `index on profiles(user_type, verification_status)`
- `index on profiles(university, user_type)`
- `index on verification_requests(user_id, status)`
- `index on verification_requests(status, submitted_at desc)`

Wallet and payments

- `unique index on wallets(user_id)`
- `index on wallet_ledger_entries(wallet_id, created_at desc)`
- `index on wallet_ledger_entries(source_type, source_id)`
- `unique index on payment_attempts(provider_reference)`
- `index on payment_attempts(user_id, purpose, created_at desc)`
- `index on payment_attempts(order_id)`
- `index on payment_attempts(status, created_at desc)`

Marketplace

- `index on marketplace_listings(seller_id, status, created_at desc)`

- `index on marketplace_listings(category_id, status, created_at desc)`
- `index on marketplace_listings(item_type, status)`
- `GIN index on search_vector`

Orders

- `index on orders(buyer_id, created_at desc)`
- `index on orders(status, created_at desc)`
- `index on orders(payment_status, created_at desc)`
- `index on order_items(order_id)`
- `index on order_items(seller_id, status, created_at desc)`

Messaging

- `index on conversations(last_message_at desc)`
- `index on conversation_participants(user_id, joined_at desc)`
- `index on messages(conversation_id, created_at desc)`
- `index on messages(sender_id, created_at desc)`

Disputes

- `index on disputes(order_id)`
- `index on disputes(status, created_at desc)`

Operational flows

Registration flow

1. Create auth account
2. Create `profiles` row
3. Auto-create `wallets` row
4. Default user should be unverified and not premium

Student KYC flow

1. Student submits KYC
2. Create `verification_requests` row with `status = pending`
3. Upload files and create `verification_documents`
4. Super admin reviews
5. On approval:

- `profiles.is_verified = true`
- `profiles.verification_status = verified`

6. On rejection:

- keep `profiles.is_verified = false`
- save rejection reason

Wallet funding flow

1. Create `payment_attempts` for wallet deposit
2. Initialize Paystack
3. Verify payment on backend or webhook
4. Insert credit entry into `wallet_ledger_entries`
5. Update wallet balance

Marketplace checkout flow

1. Create order draft
2. Create `payment_attempts`
3. Verify payment server-side
4. Mark order paid
5. Copy listing values into `order_items`
6. Record status in `order_status_history`

Wallet checkout flow

1. Check wallet balance
2. Insert debit ledger entry
3. Update wallet balance
4. Mark order paid and processing

Premium subscription flow

1. Admin defines plans in `subscription_plans`
2. User selects plan
3. Create `payment_attempts`
4. Verify payment
5. Create or update `subscriptions`
6. Set `profiles.is_premium = true`

Messaging flow

1. Open or create `conversations`
2. Add users to `conversation_participants`
3. Insert `messages`
4. Track read state via `last_read_message_id`

Dispute flow

1. User opens dispute
2. Create `disputes` row
3. Admin reviews using order data, message history, and payment records
4. Resolve with refund, credit, rejection, or closure

Rollout phases

Phase 1

Build the minimum persistent commerce layer:

- `profiles`
- `wallets`
- `wallet_ledger_entries`
- `payment_attempts`
- `marketplace_categories`
- `marketplace_listings`
- `orders`
- `order_items`
- `order_status_history`

Phase 2

Add verification and admin control:

- `verification_requests`
- `verification_documents`
- `payout_accounts`
- `admin_audit_logs`
- `system_settings`

Phase 3

Add messaging:

- `conversations`
- `conversation_participants`
- `messages`
- `message_attachments`

Phase 4

Add premium billing:

- `subscription_plans`
- `subscriptions`

Phase 5

Add dispute and moderation maturity:

- `disputes`

Recommended first SQL migration order

Create tables in this order:

1. `profiles`
2. `wallets`
3. `verification_requests`
4. `verification_documents`
5. `payout_accounts`
6. `subscription_plans`
7. `subscriptions`
8. `marketplace_categories`
9. `marketplace_listings`
10. `listing_media`
11. `orders`
12. `order_items`
13. `order_status_history`
14. `payment_attempts`
15. `wallet_ledger_entries`
16. `conversations`
17. `conversation_participants`

18. `messages`
19. `message_attachments`
20. `disputes`
21. `system_settings`
22. `admin_audit_logs`

Notes for this project

This schema is based on the current CampusLink frontend behavior:

- student, expert, and institution accounts
- manual verification and student KYC
- marketplace listings and category pages
- wallet deposits, withdrawals, and payments
- Paystack billing
- premium plan administration
- checkout, cart, and order flow
- future persistent messaging and dispute review

Next recommended deliverables

After this document, the next best outputs are:

1. SQL schema migration files
2. ER diagram
3. Supabase table and policy design
4. API contract for auth, billing, orders, wallet, and marketplace